In [ ]:
```
## Question #02
from copy import deepcopy
def print_board(board):
    for row in board:
        print(" ".join(str(num) for num in row))
    print()


def get_domains(board):
    """Initialize domains for each cell"""
    domains = {}
    for r in range(9):
        for c in range(9):
            if board[r][c] == 0:
                domains[(r, c)] = set(range(1, 10))
            else:
                domains[(r, c)] = {board[r][c]}
    return domains


def get_neighbors(cell):
    """Return all neighbors (row, column, box)"""
    r, c = cell
    neighbors = set()

    # Row and column
    for i in range(9):
        neighbors.add((r, i))
        neighbors.add((i, c))

    # 3x3 box
    start_r, start_c = 3 * (r // 3), 3 * (c // 3)
    for i in range(3):
        for j in range(3):
            neighbors.add((start_r + i, start_c + j))

    neighbors.remove(cell)
    return neighbors

def revise(domains, xi, xj):
    revised = False

    if len(domains[xj]) == 1:
        val = next(iter(domains[xj]))
        if val in domains[xi]:
            domains[xi].remove(val)
            revised = True

    return revised


def ac3(domains):
    queue = [(xi, xj) for xi in domains for xj in get_neighbors(xi)]
```

```python
    while queue:
        xi, xj = queue.pop(0)

        if revise(domains, xi, xj):
            if len(domains[xi]) == 0:
                return False

            for xk in get_neighbors(xi):
                if xk != xj:
                    queue.append((xk, xi))

    return True

def is_complete(domains):
    return all(len(v) == 1 for v in domains.values())


def select_unassigned_variable(domains):
    """MRV heuristic"""
    return min([v for v in domains if len(domains[v]) > 1],
               key=lambda x: len(domains[x]))


def forward_check(domains, var):
    """Remove assigned value from neighbors"""
    value = next(iter(domains[var]))

    for neighbor in get_neighbors(var):
        if value in domains[neighbor]:
            domains[neighbor].remove(value)
            if len(domains[neighbor]) == 0:
                return False
    return True

backtrack_calls = 0
failures = 0


def backtrack(domains):
    global backtrack_calls, failures
    backtrack_calls += 1

    if is_complete(domains):
        return domains

    var = select_unassigned_variable(domains)

    for value in domains[var].copy():
        new_domains = deepcopy(domains)
        new_domains[var] = {value}

        if forward_check(new_domains, var) and ac3(new_domains):
            result = backtrack(new_domains)
            if result:
                return result
```

```
    failures += 1
    return None


def solve(board):
    global backtrack_calls, failures

    backtrack_calls = 0
    failures = 0

    domains = get_domains(board)

    # Initial AC-3 pruning
    ac3(domains)

    result = backtrack(domains)

    if result:
        solved_board = [
            [next(iter(result[(r, c)])) for c in range(9)]
            for r in range(9)
        ]
        return solved_board, backtrack_calls, failures

    return None, backtrack_calls, failures


def read_board_from_file(filename):
    board = []
    with open(filename, 'r') as file:
        for line in file:
            row = [int(ch) for ch in line.strip()]
            board.append(row)
    return board


if __name__ == "__main__":
    filename = input("Enter Sudoku file name: ")

    board = read_board_from_file(filename)

    print("Original Board:")
    print_board(board)

    solution, calls, fails = solve(board)

    if solution:
        print("Solved Board:")
        print_board(solution)
        print("Backtrack Calls:", calls)
        print("Failures:", fails)
    else:
        print("No solution found.")
```

In [ ]: